NanoClaw vs OpenClaw — Architecture Comparison

NanoClaw vs OpenClaw — Deep Architecture Comparison

> Diagrams use Mermaid — paste any block into mermaid.live to render interactively.

---

1. Top-Level Philosophy

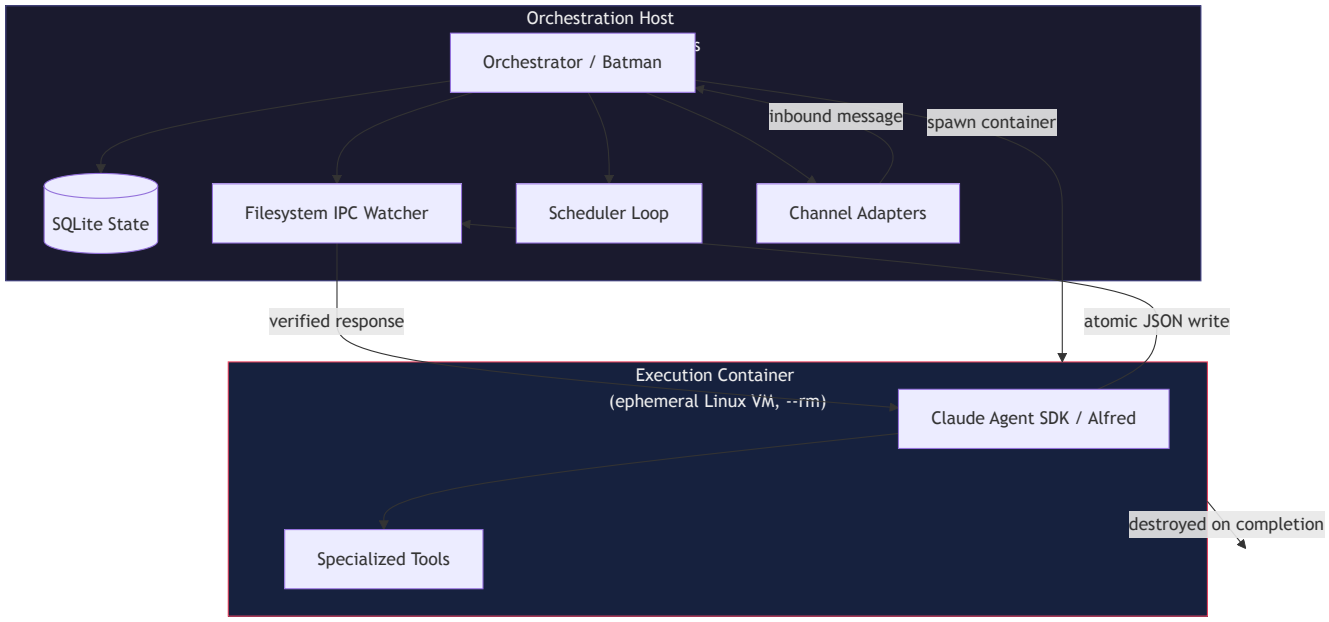
Dimension	NanoClaw	OpenClaw
Core premise	Agent is dangerous, must be caged	Agent is powerful, must be orchestrated
Codebase size	~4,000 lines (intentional)	Enterprise monorepo, 100k+ lines
Security model	OS-level / hardware isolation	Application-layer + operator trust
Extensibility	Static code transformation	Runtime plugin registry (ClawHub)
Deployment target	Single-user, security-critical	Teams, cloud, Kubernetes
Stars / adoption	Niche, growing	215,000+ stars
License	—	MIT

---

2. High-Level Architecture

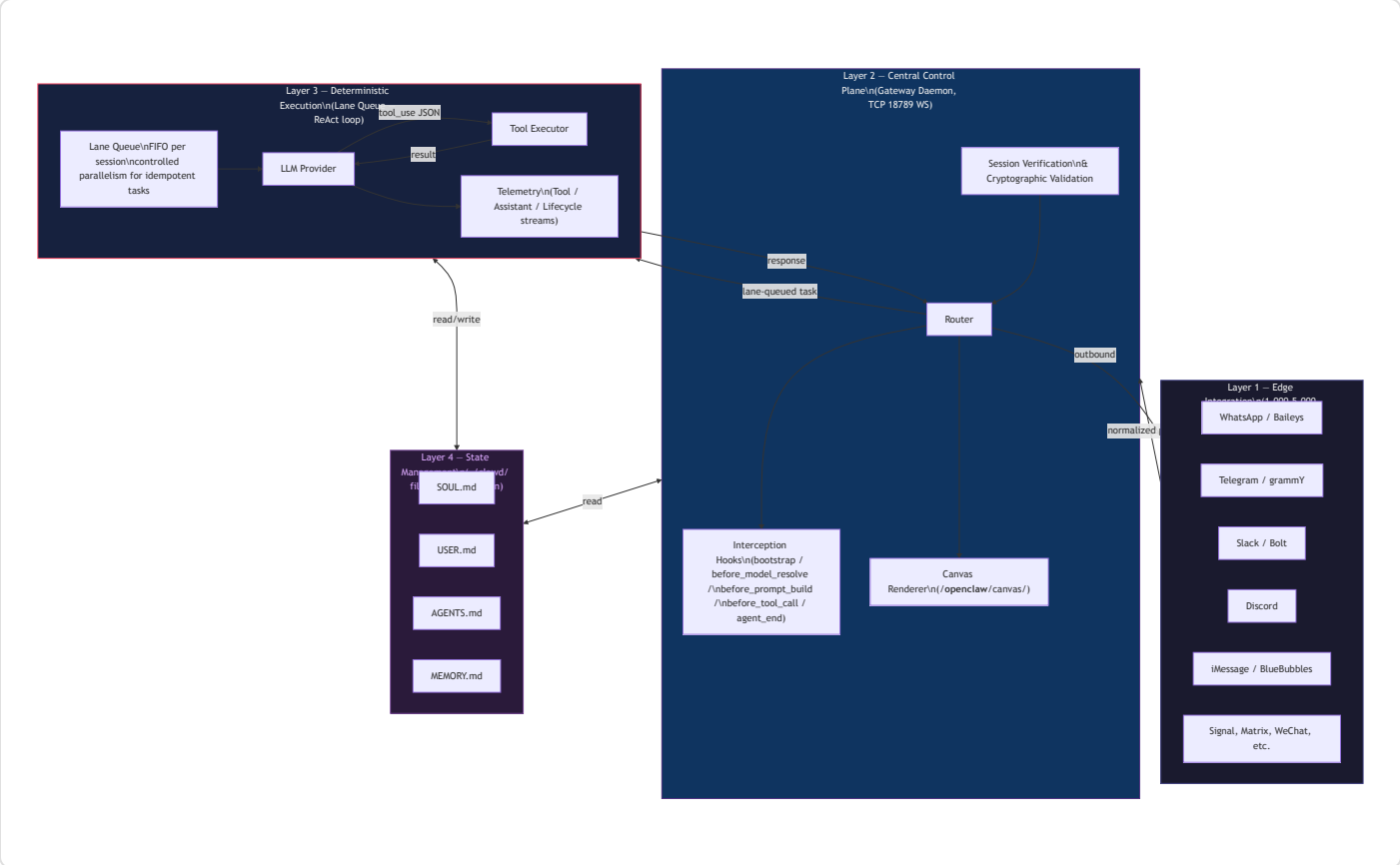
NanoClaw

NanoClaw decomposes into a small set of named roles: the **Orchestrator (Batman)** that never touches the Claude API directly, the **Filesystem IPC Watcher** that mediates all agent-to-host communication, the **Scheduler Loop** that manages container lifecycle, and the **Channel Adapters** that normalize inbound messages — all running on a Node.js host process. The agent itself executes inside an ephemeral, network-isolated Linux VM and communicates exclusively through atomic JSON files on a shared mount.



OpenClaw

OpenClaw is structured as four named layers. **Layer 1** is the Edge Integration tier — one adapter per channel (WhatsApp/Baileys, Telegram/grammY, Slack/Bolt, Discord, iMessage/BlueBubbles, and others), each 1,000–5,000 lines. **Layer 2** is the Central Control Plane — a Gateway daemon on TCP 18789 with a Router, Session Verification, Interception Hooks, and a Canvas Renderer. **Layer 3** is the Deterministic Execution tier — a Lane Queue (FIFO per session) feeding a ReAct loop with LLM Provider, Tool Executor, and Telemetry streams. **Layer 4** is the State Management tier — file-based Markdown documents under `~/clawd/` (SOUL.md, USER.md, AGENTS.md, MEMORY.md).

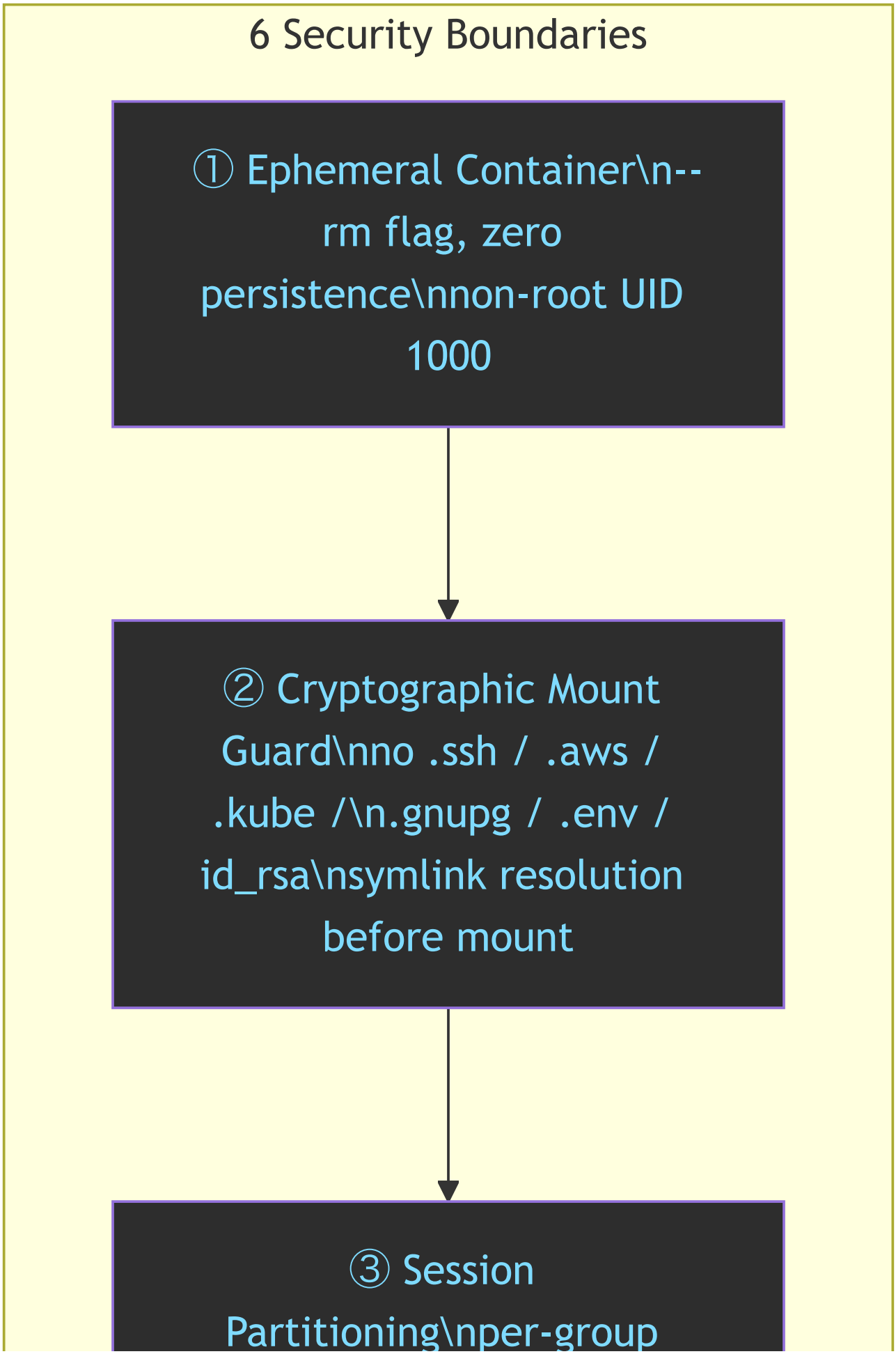


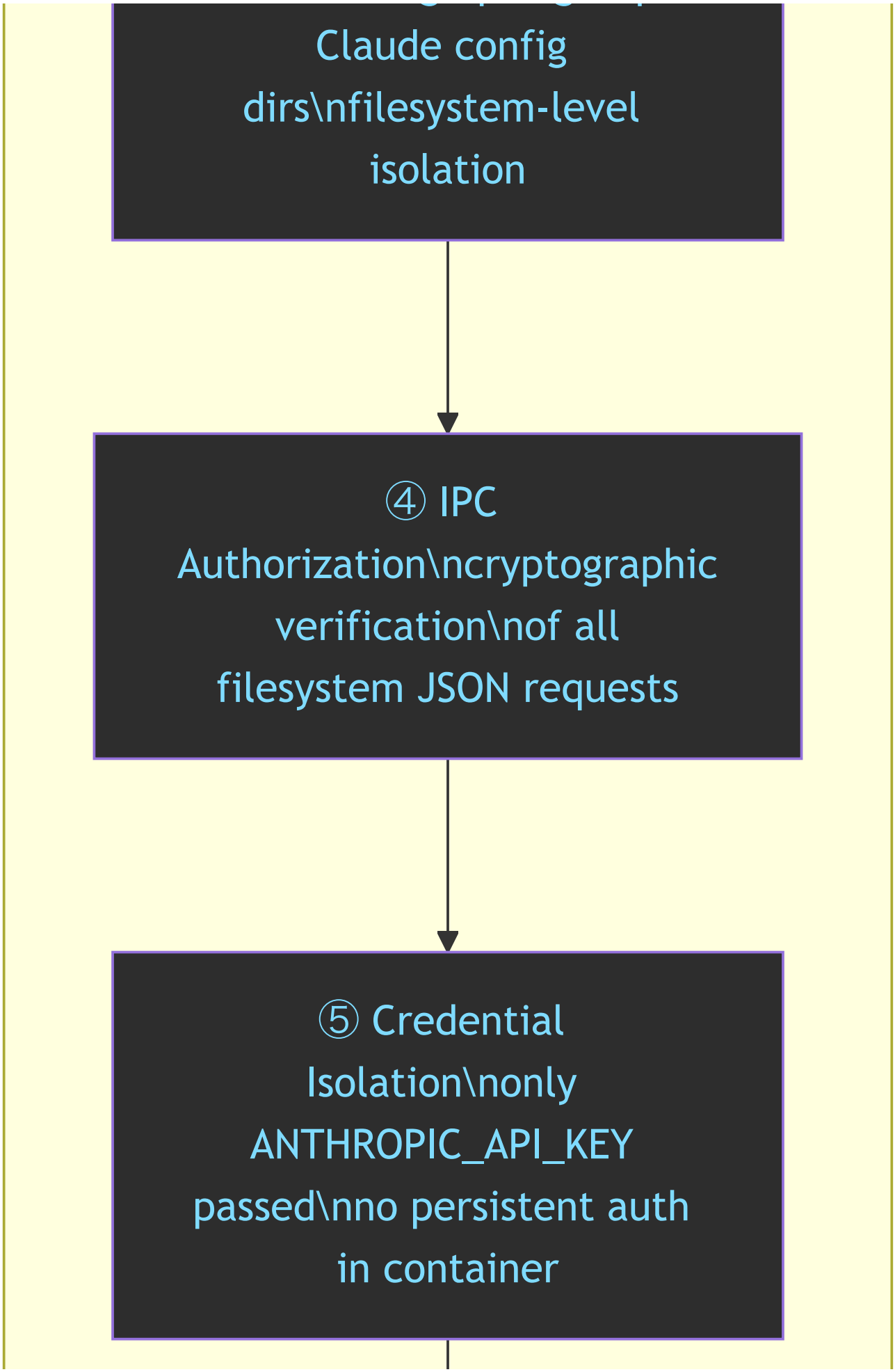
---

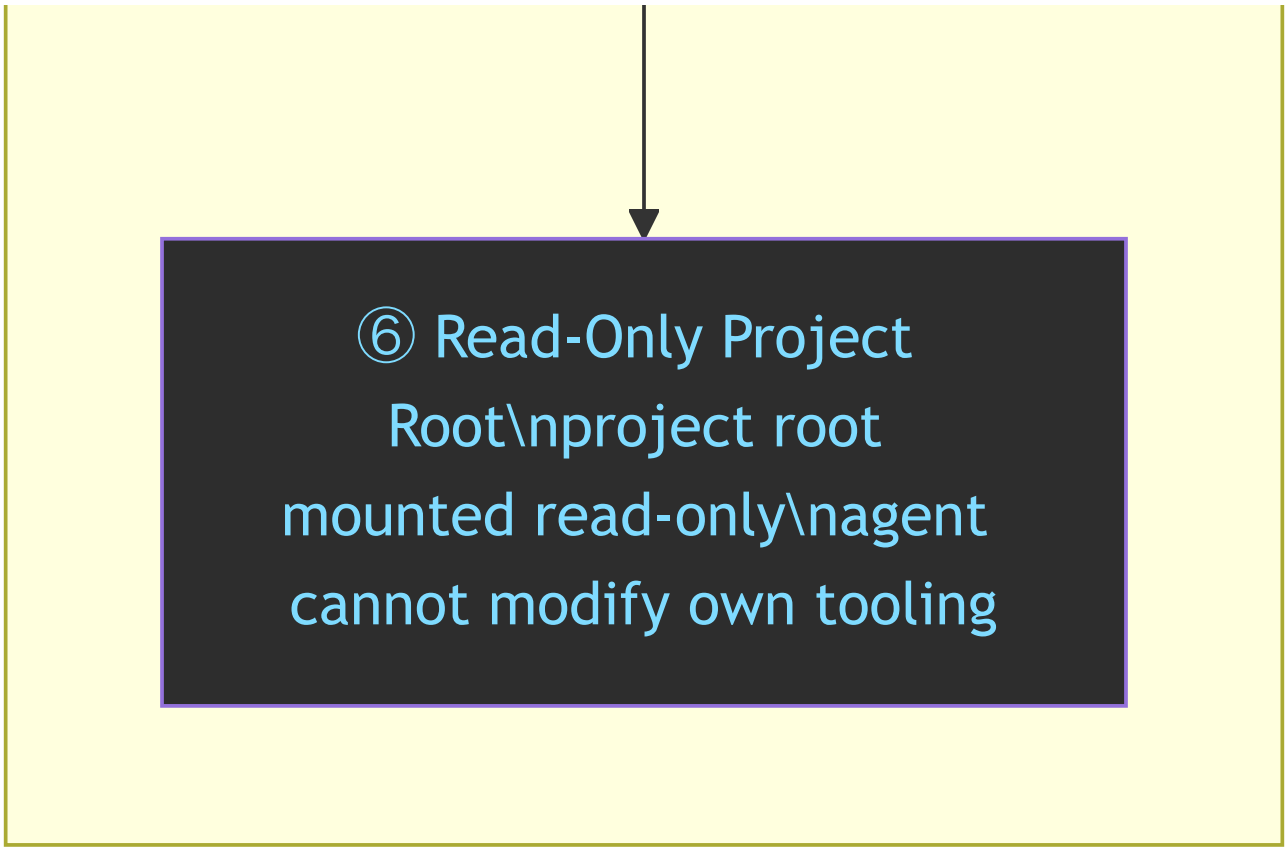
3. Security Architecture

NanoClaw — Defense in Depth (OS-level)

NanoClaw enforces **six** distinct security boundaries. B1 now runs the container agent under non-root UID 1000 in addition to the `--rm` flag. B2's cryptographic mount guard blocks an expanded blocklist that includes `.gnupg`, `.env`, and `id_rsa` in addition to `.ssh`, `.aws`, and `.kube`. B6 is a new boundary: the project root is mounted read-only, preventing the agent from modifying its own tooling or orchestration code.

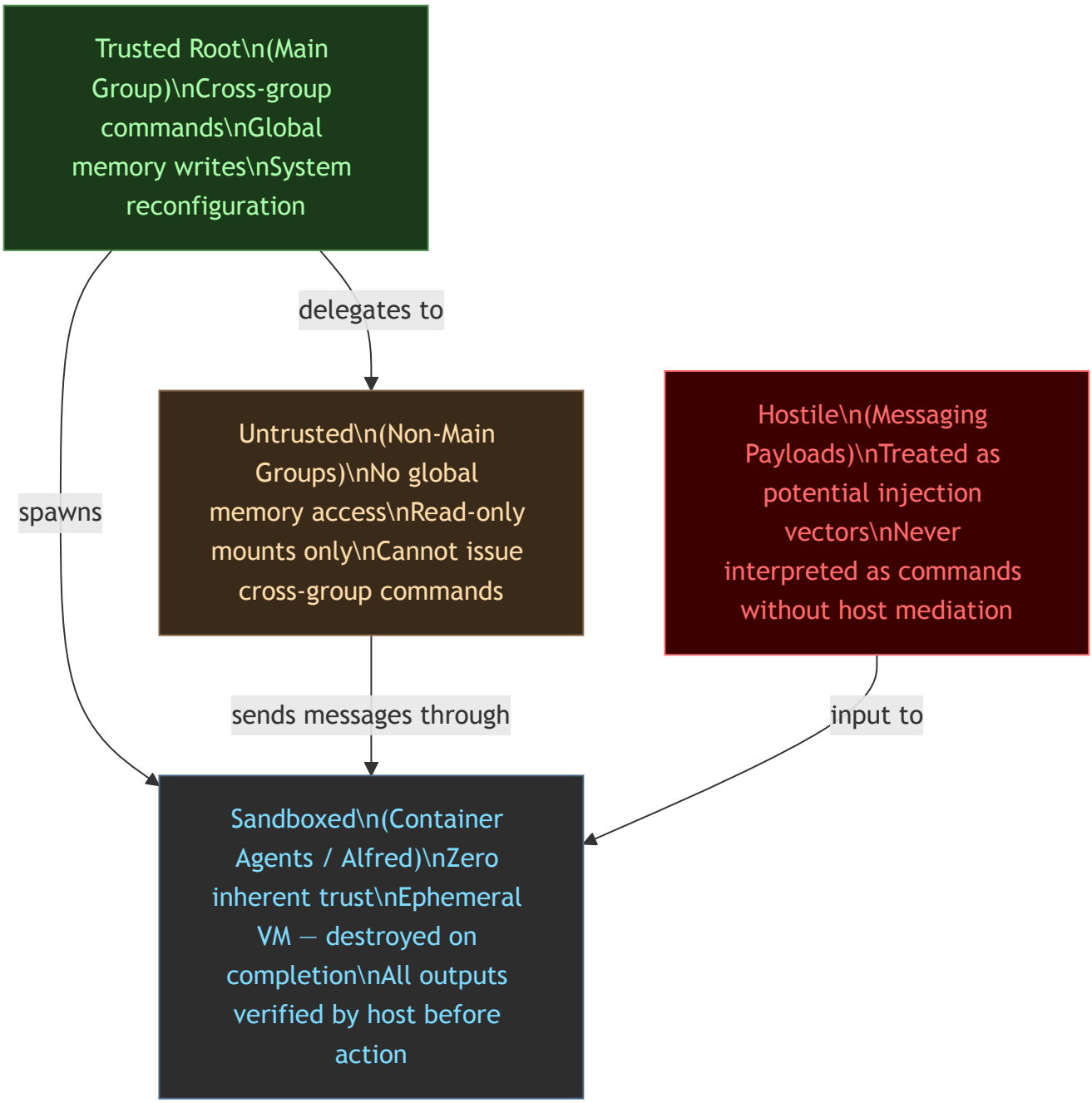

6 Security Boundaries
① Ephemeral Container\n--rm flag, zero persistence\nnon-root UID 1000
② Cryptographic Mount Guard\nno .ssh / .aws / .kube /\n.gnupg / .env / id_rsa\nsymlink resolution before mount
③ Session Partitioning\nper-group


Claude config dirs\nfilesystem-level isolation
④ IPC Authorization\ncryptographic verification\nof all filesystem JSON requests
⑤ Credential Isolation\nonly ANTHROPIC_API_KEY passed\nno persistent auth in container

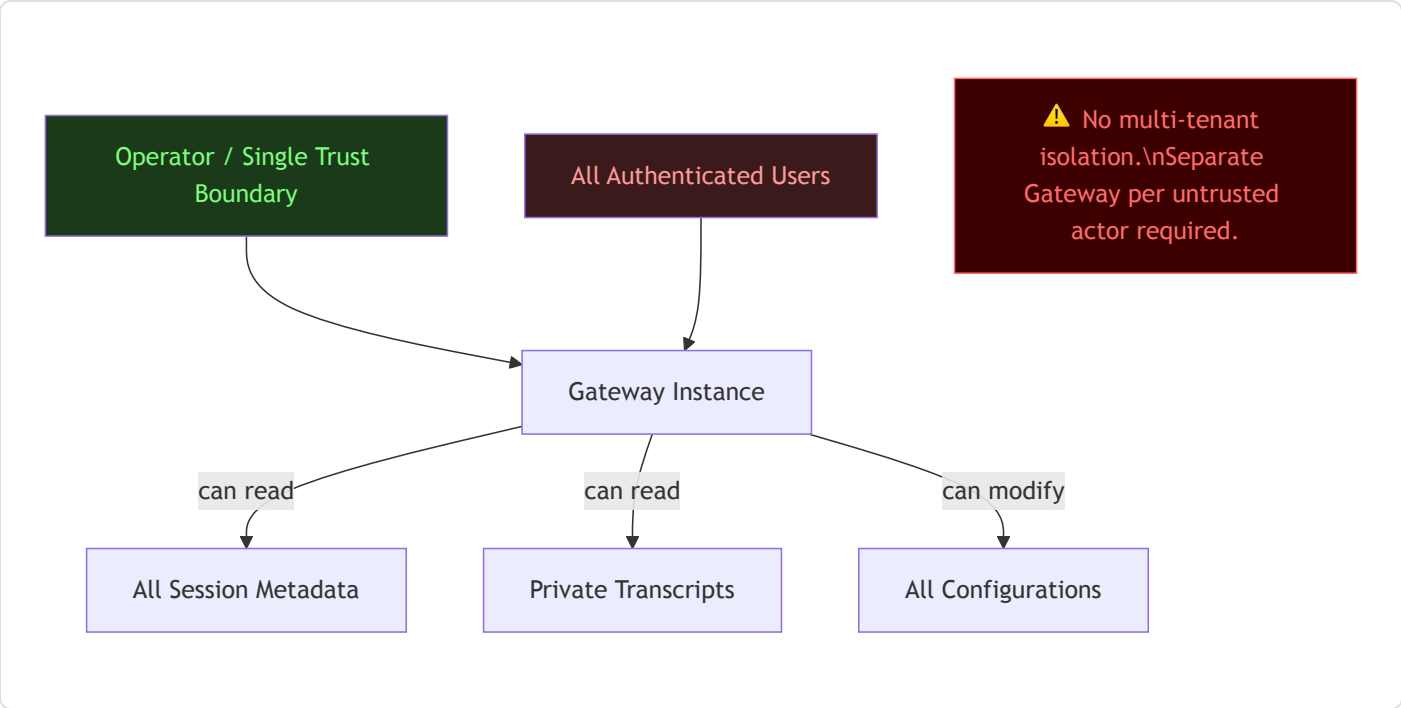


NanoClaw — Trust Taxonomy

NanoClaw applies a four-tier trust hierarchy that is enforced structurally — not by policy flags — because each tier is implemented at a different system boundary.


Trusted Root\n(Main Group)\nCross-group commands\nGlobal memory writes\nSystem reconfiguration
delegates to
spawns
Untrusted\n(Non-Main Groups)\nNo global memory access\nRead-only mounts only\nCannot issue cross-group commands
Hostile\n(Messaging Payloads)\nTreated as potential injection vectors\nNever interpreted as commands without host mediation
sends messages through
input to
Sandboxed\n(Container Agents / Alfred)\nZero inherent trust\nEphemeral VM — destroyed on completion\nAll outputs verified by host before action

OpenClaw — Application-Layer Trust Model



Vulnerability Comparison

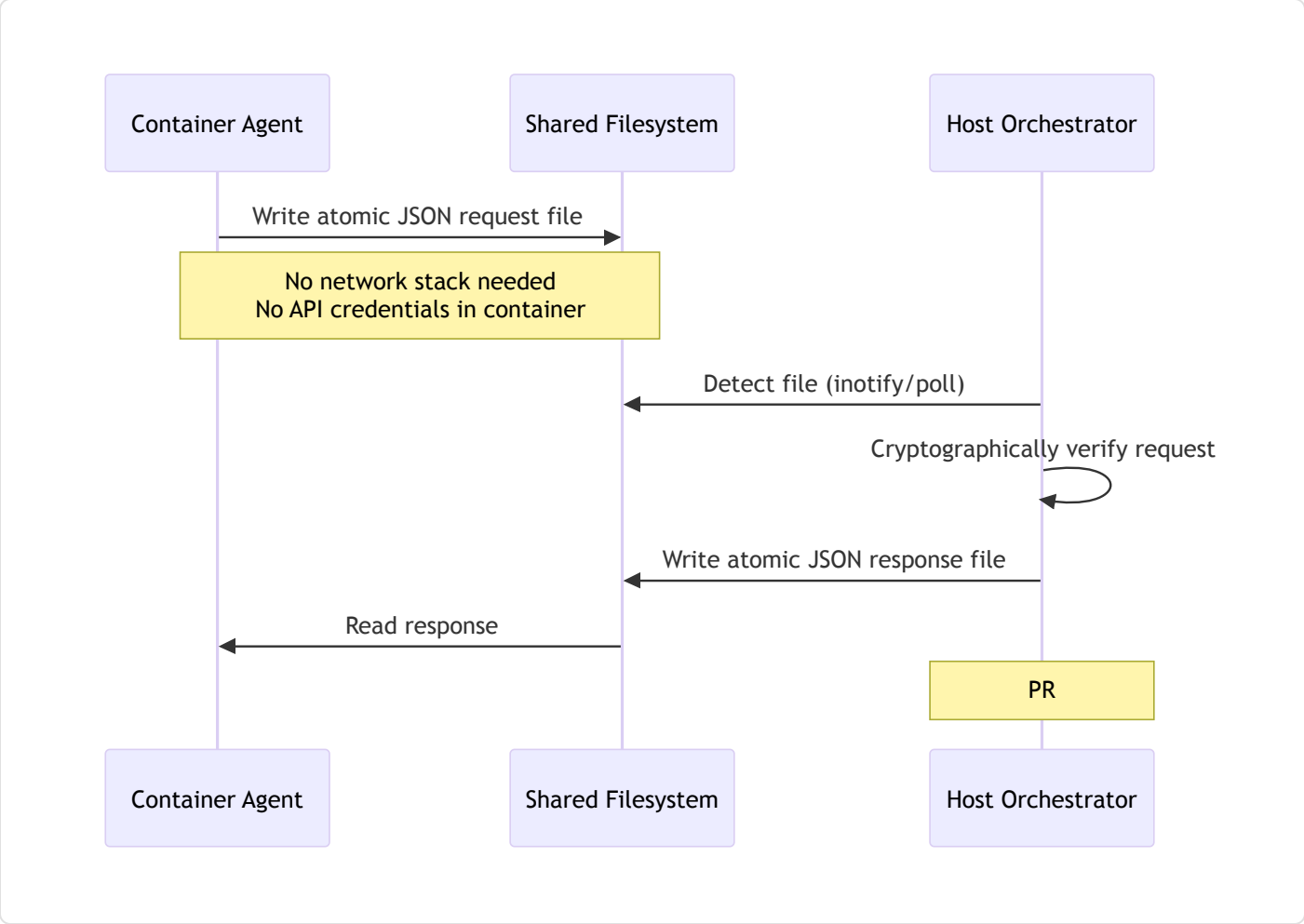
Attack Surface	NanoClaw	OpenClaw
Prompt injection → host escape	Blocked by VM boundary (UID 1000 + ephemeral container)	Application-layer mitigations only
Cross-session data leak	Blocked by filesystem partitioning	Possible — single trust boundary
Credential exposure in container	Only API key, no persistent auth	Configurable, defaults vary
Network attack surface	None — containers have no network stack	TCP/18789 WebSocket, loopback default
Supply chain (skills)	Static merge, no runtime exec of skill code (see Section 6)	7.1% of ClawHub skills mishandle secrets; 283–341 confirmed credential stealers (see Section 6)
Multi-tenant	Not supported (by design — single user)	Not supported (explicit doc disclaimer)
Memory accumulation (CWE-770)	N/A	Known bug in `command-queue.ts` — LaneState Map never cleaned, causes OOM
WebSocket hijacking	N/A	Session fixation via CSRF → persistent bidirectional access

Attack Surface	NanoClaw	OpenClaw
Network misconfiguration	N/A	0.0.0.0 binding → RCE within minutes of public exposure

---

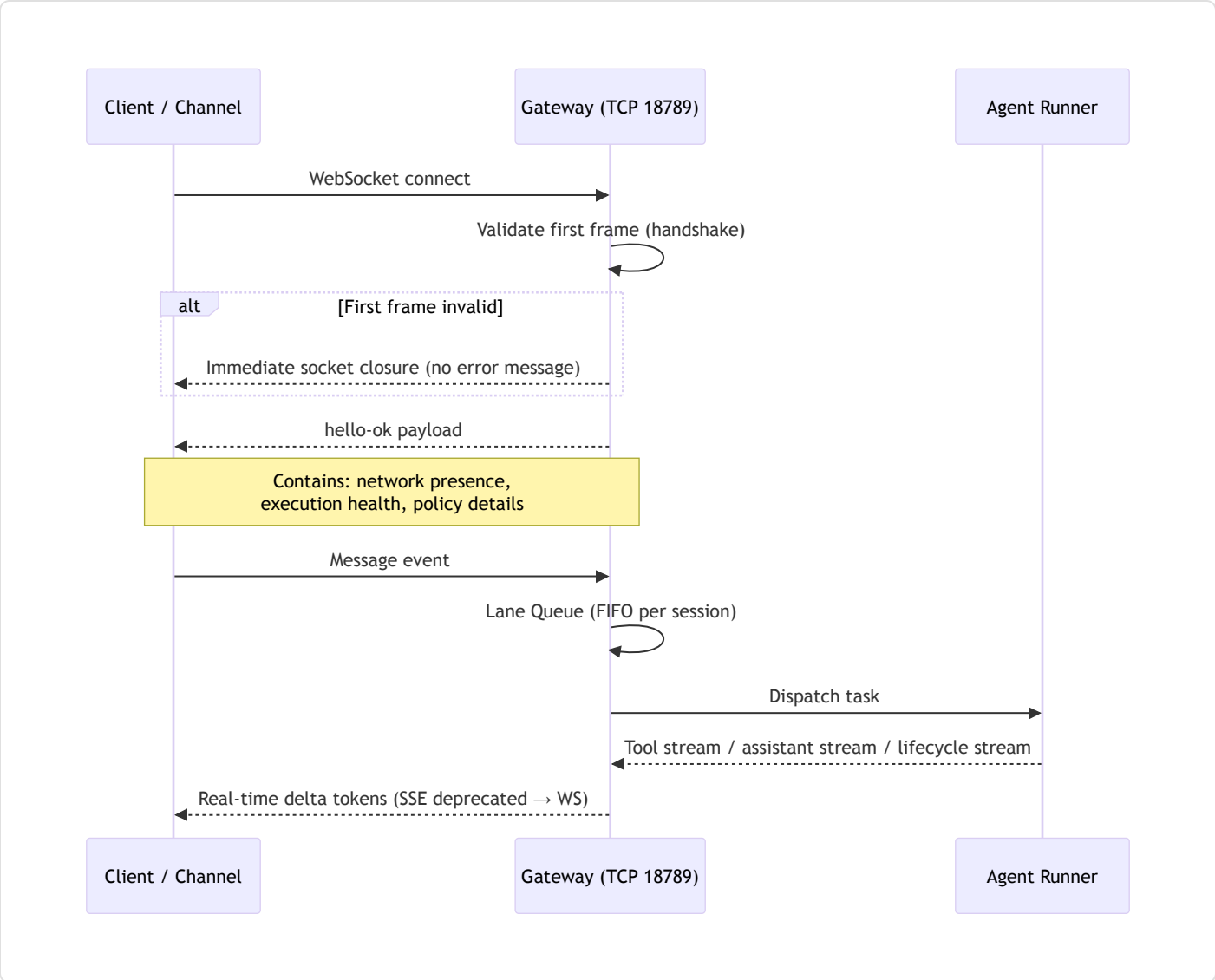
4. Inter-Process Communication

NanoClaw — Filesystem IPC



Properties: - Zero network exposure inside container - Atomic writes prevent stream injection - Request forgery blocked by crypto verification - Host never writes back without verification

OpenClaw — WebSocket Transport

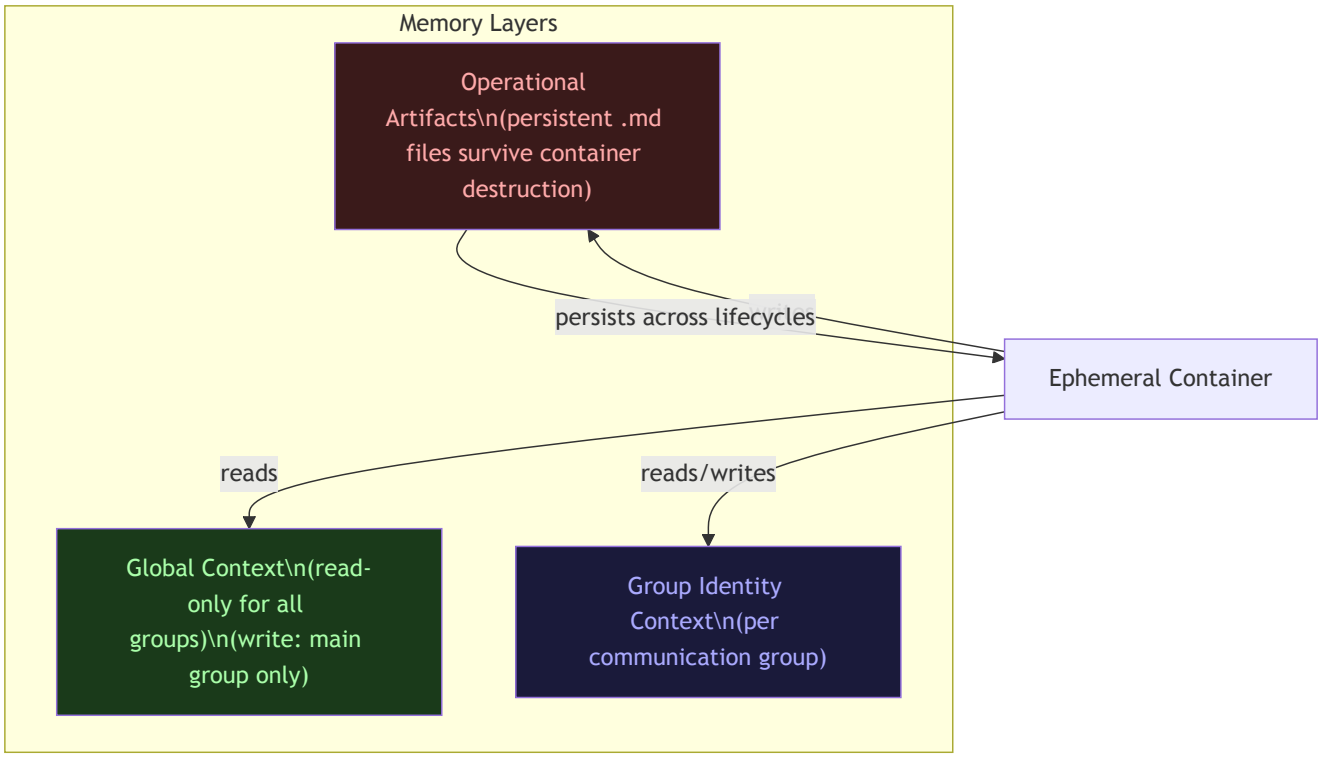


Properties: - Bidirectional, real-time - Strict FIFO per lane prevents race conditions - Invalid first frames result in immediate silent socket closure (no error payload leaked) - Unencrypted by default — TLS is operator responsibility - Session fixation risk on hijack

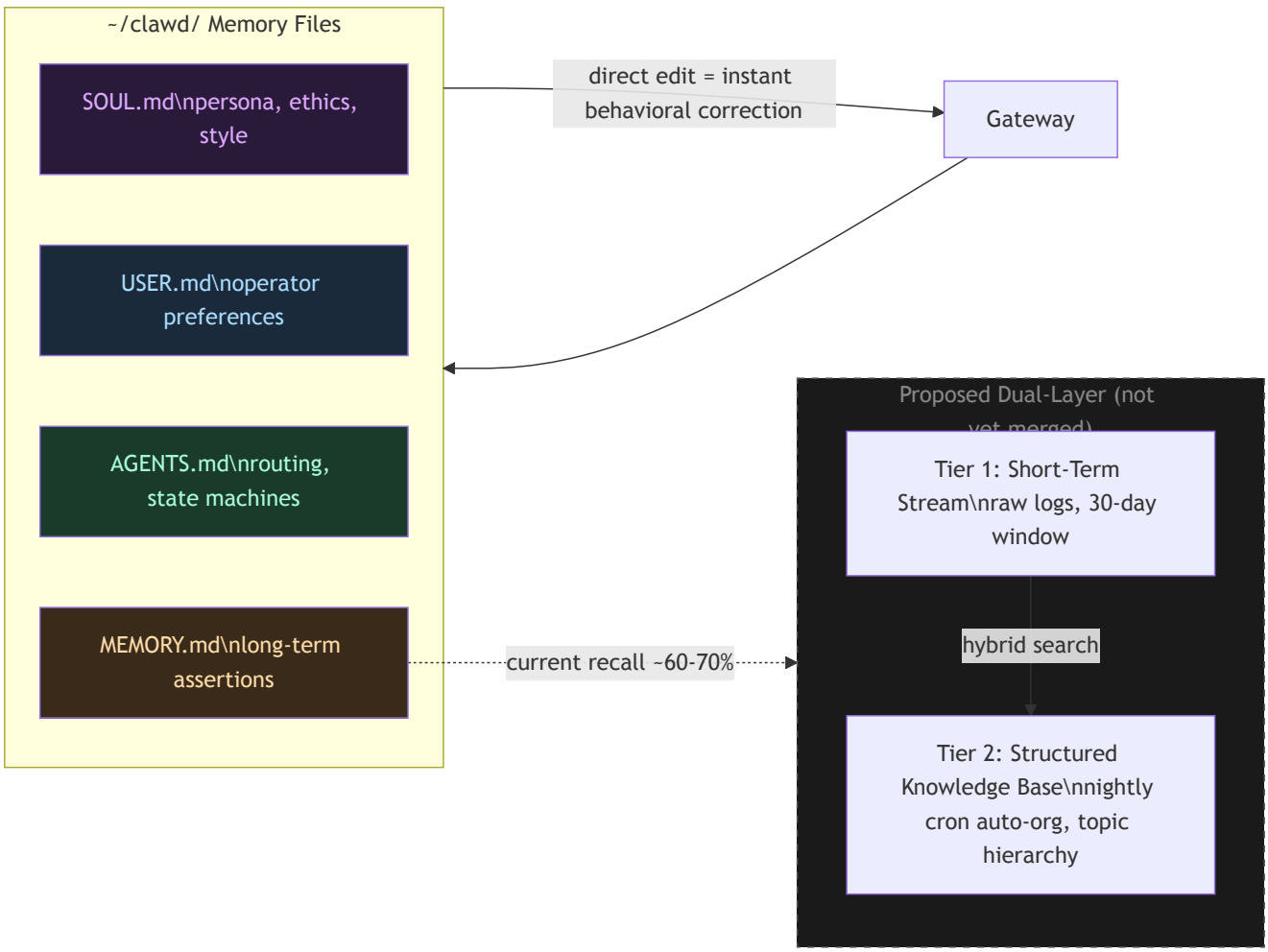
---

5. Memory Architecture

NanoClaw — Three-Layer Memory

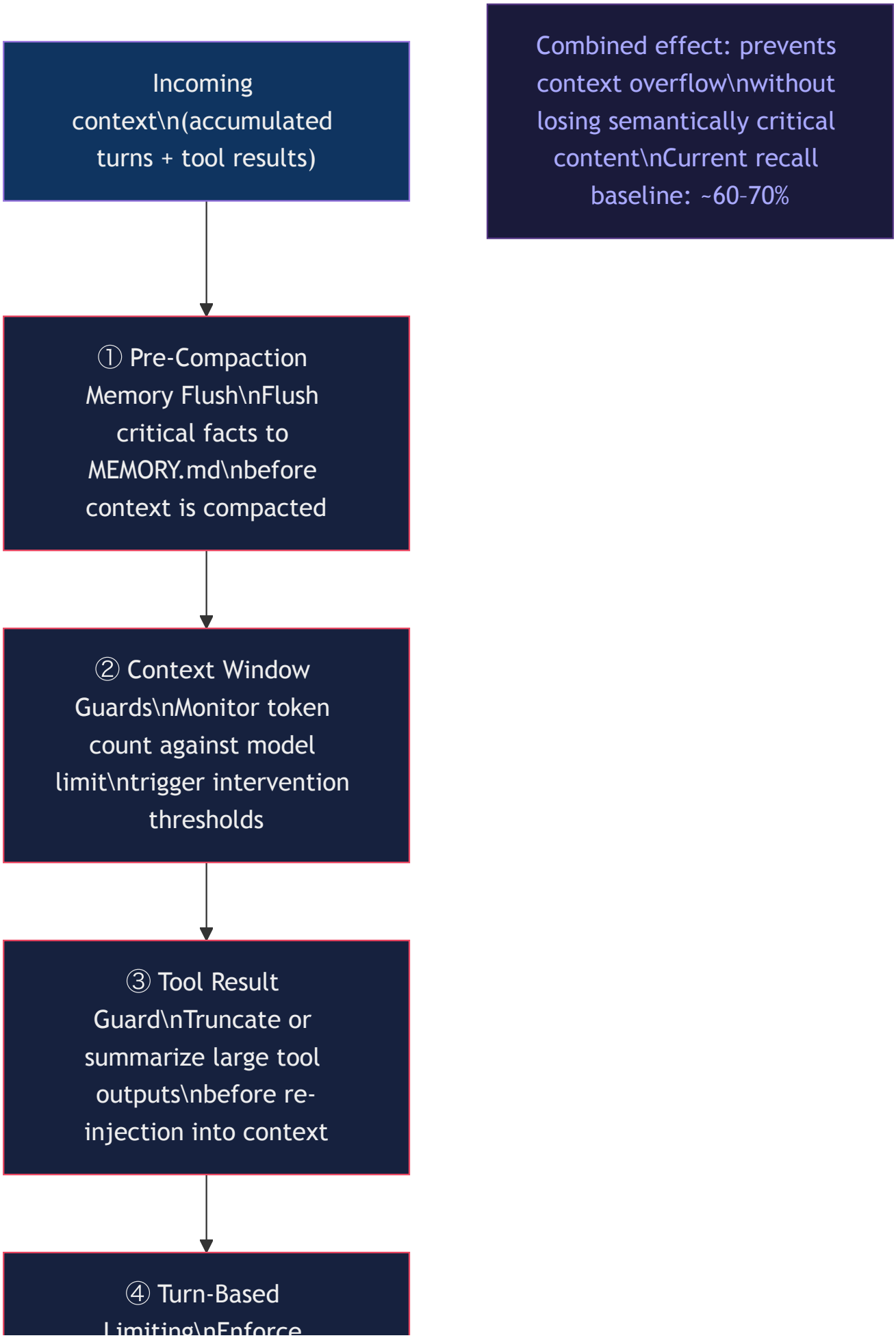


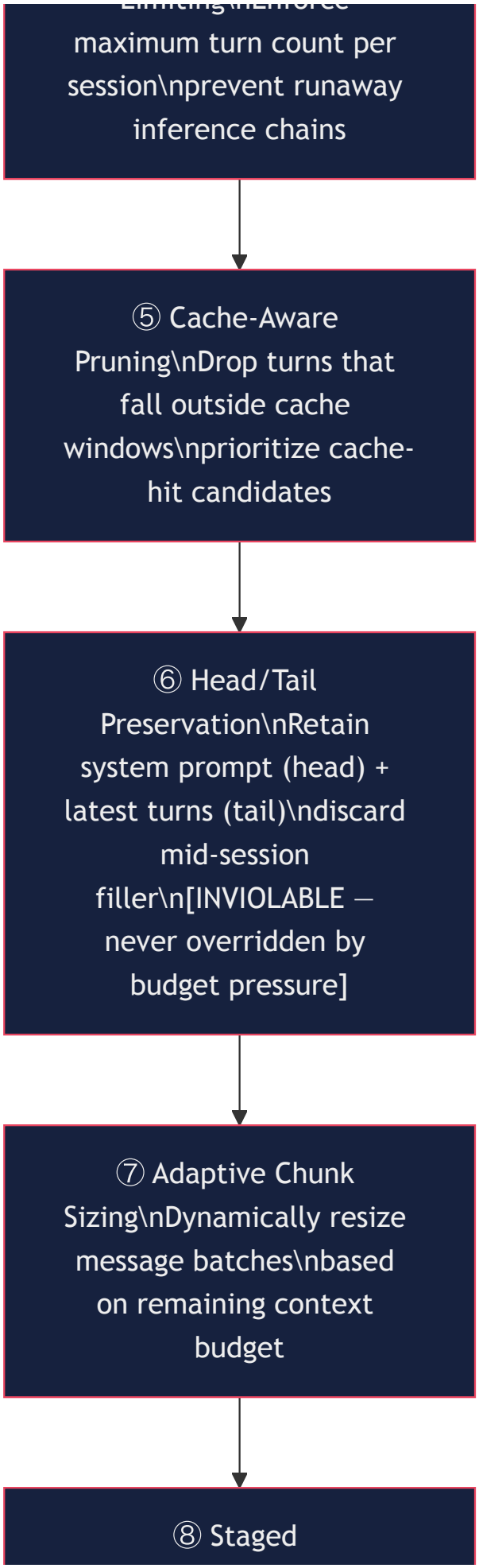
OpenClaw — File-Based Markdown Memory

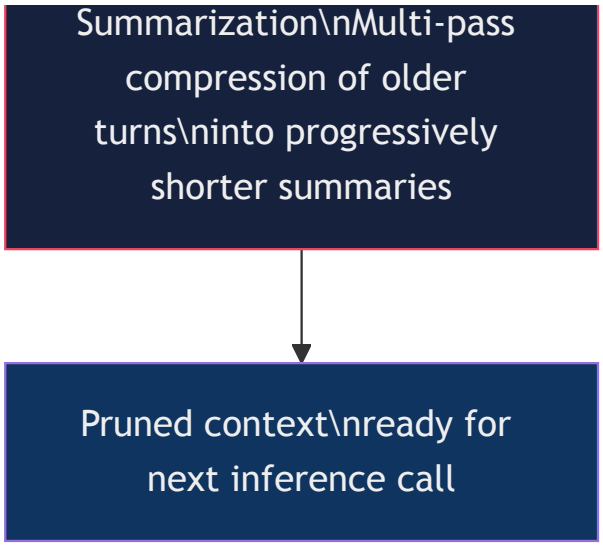


OpenClaw — Context Management Strategy

Because OpenClaw runs long-lived agent sessions over WebSocket, accumulated context (turns + large tool results) eventually threatens the model's context window. OpenClaw addresses this with an eight-stage pipeline. Stages S1 through S8 are applied in sequence to every context before the next inference call. The Head (system prompt) and Tail (latest turns) are treated as **inviolable** by the Head/Tail Preservation stage — they are never discarded regardless of budget pressure; all pruning targets mid-session filler between them.


Incoming context\n(accumulated turns + tool results)
① Pre-Compaction Memory Flush\nFlush critical facts to MEMORY.md\nbefore context is compacted
② Context Window Guards\nMonitor token count against model limit\ntrigger intervention thresholds
③ Tool Result Guard\nTruncate or summarize large tool outputs\nbefore re-injection into context
④ Turn-Based
Combined effect: prevents context overflow\nwithout losing semantically critical content\nCurrent recall baseline: ~60-70%

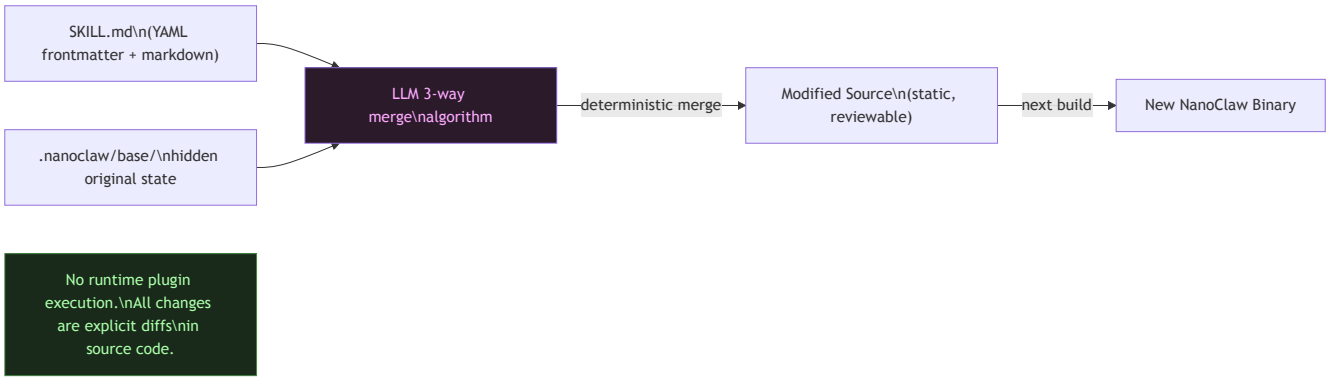

maximum turn count per session\nprevent runaway inference chains
⑤ Cache-Aware Pruning\nDrop turns that fall outside cache windows\nprioritize cache-hit candidates
⑥ Head/Tail Preservation\nRetain system prompt (head) + latest turns (tail)\ndiscard mid-session filler\n[INVIOLABLE — never overridden by budget pressure]
⑦ Adaptive Chunk Sizing\nDynamically resize message batches\nbased on remaining context budget
⑧ Staged



---

6. Extensibility / Skills Engine

NanoClaw — Static Code Transformation



Key properties: - Skills are merged *into* the codebase, not loaded at runtime - Every integration is an auditable source diff - Codebase stays small (skills don't accumulate as runtime deps) - Follows Anthropic Agent Skills open standard

NanoClaw — Channel Registry Interface

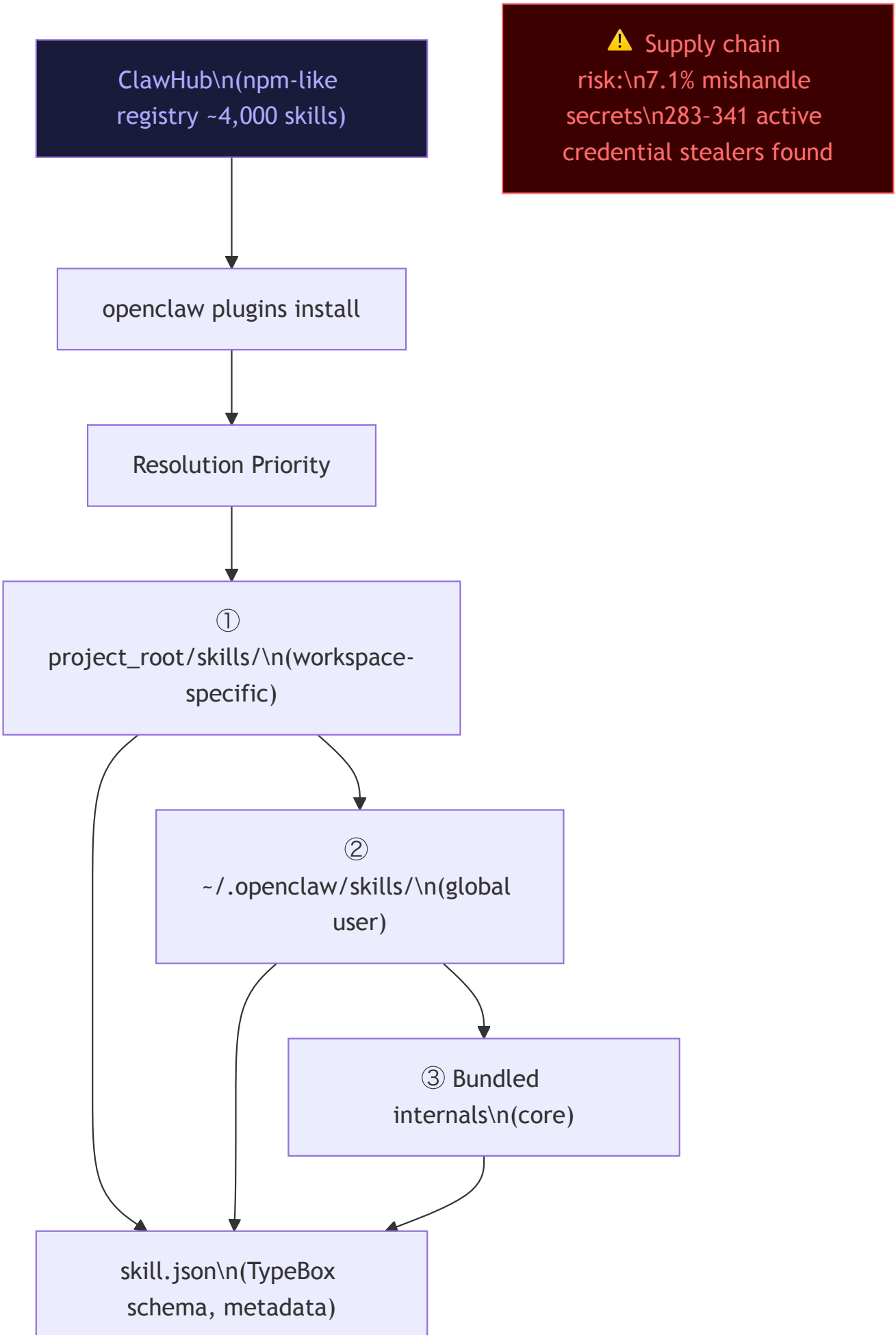
Each channel adapter in NanoClaw is registered through a factory pattern that enforces a uniform five-method interface. This means any new channel integration is automatically subject to the same lifecycle and ownership guarantees without requiring changes to the orchestrator. The interface is:

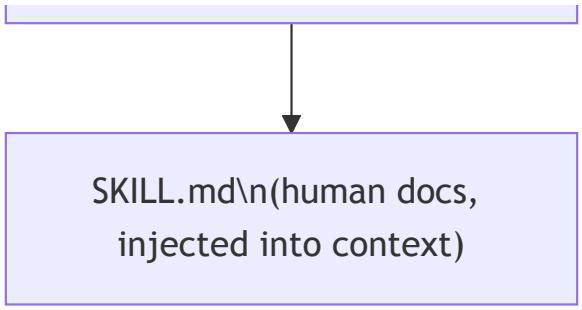
```typescript
interface ChannelAdapter {
  connect(): Promise<void>;
  sendMessage(jid: string, content: string): Promise<void>;
  isConnected(): boolean;
  ownsJid(jid: string): boolean;
  disconnect(): Promise<void>;
}
```

- `connect` / `disconnect` — lifecycle hooks called by the Scheduler Loop
- `sendMessage` — the only outbound path; JID must be validated by the adapter before use
- `isConnected` — polled by the Orchestrator's health-check loop
- `ownsJid` — used to route inbound messages to the correct adapter when multiple channels are active simultaneously

Adapters that do not implement all five methods are rejected at registration time, preventing silent routing failures.

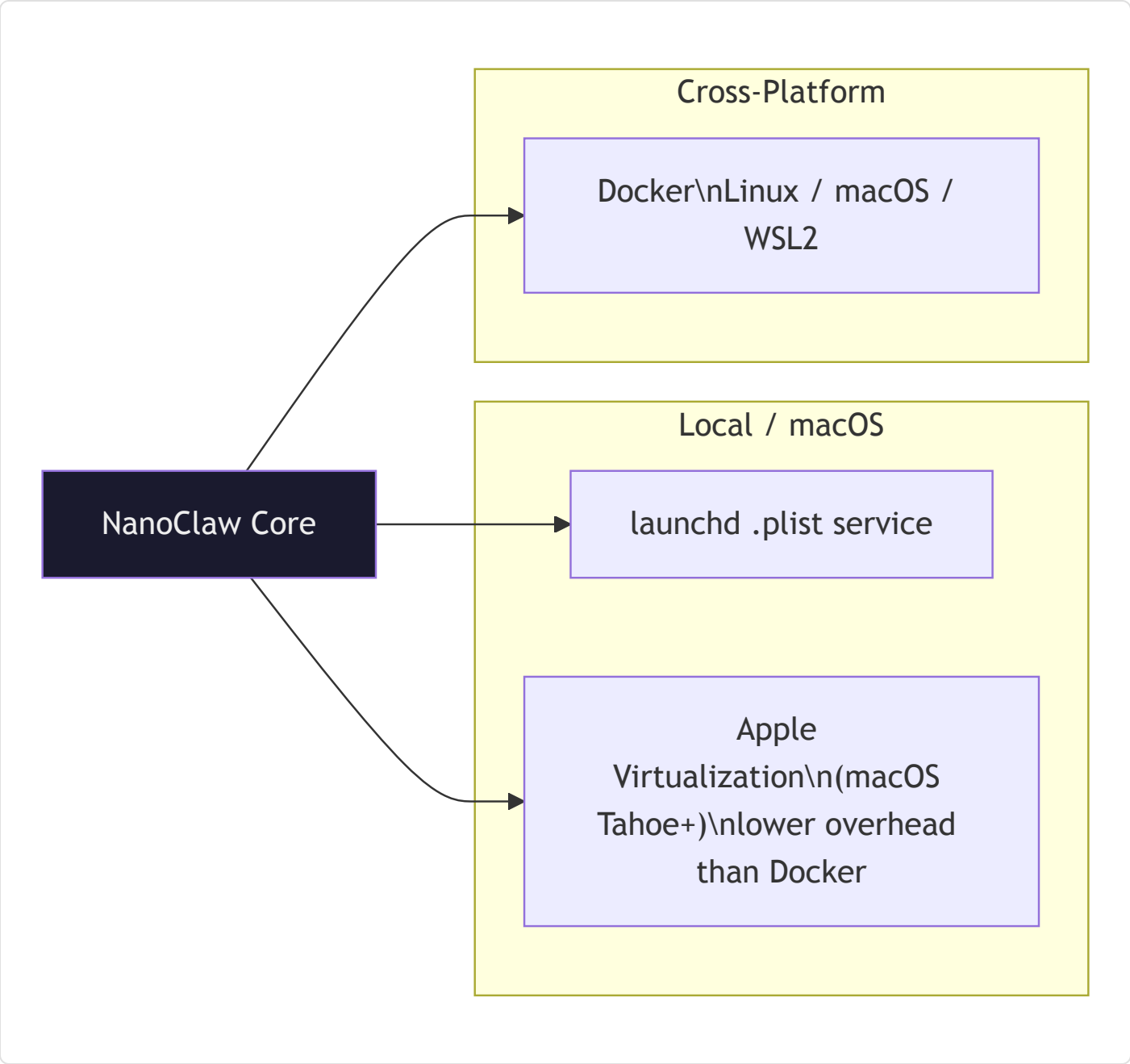
OpenClaw — Runtime Plugin Registry (ClawHub)


ClawHub\n(npm-like registry ~4,000 skills)
⚠️ Supply chain risk:\n7.1% mishandle secrets\n283–341 active credential stealers found
openclaw plugins install
Resolution Priority
① project_root/skills/\n(workspace-specific)
② ~/.openclaw/skills/\n(global user)
③ Bundled internals\n(core)
skill.json\n(TypeBox schema, metadata)

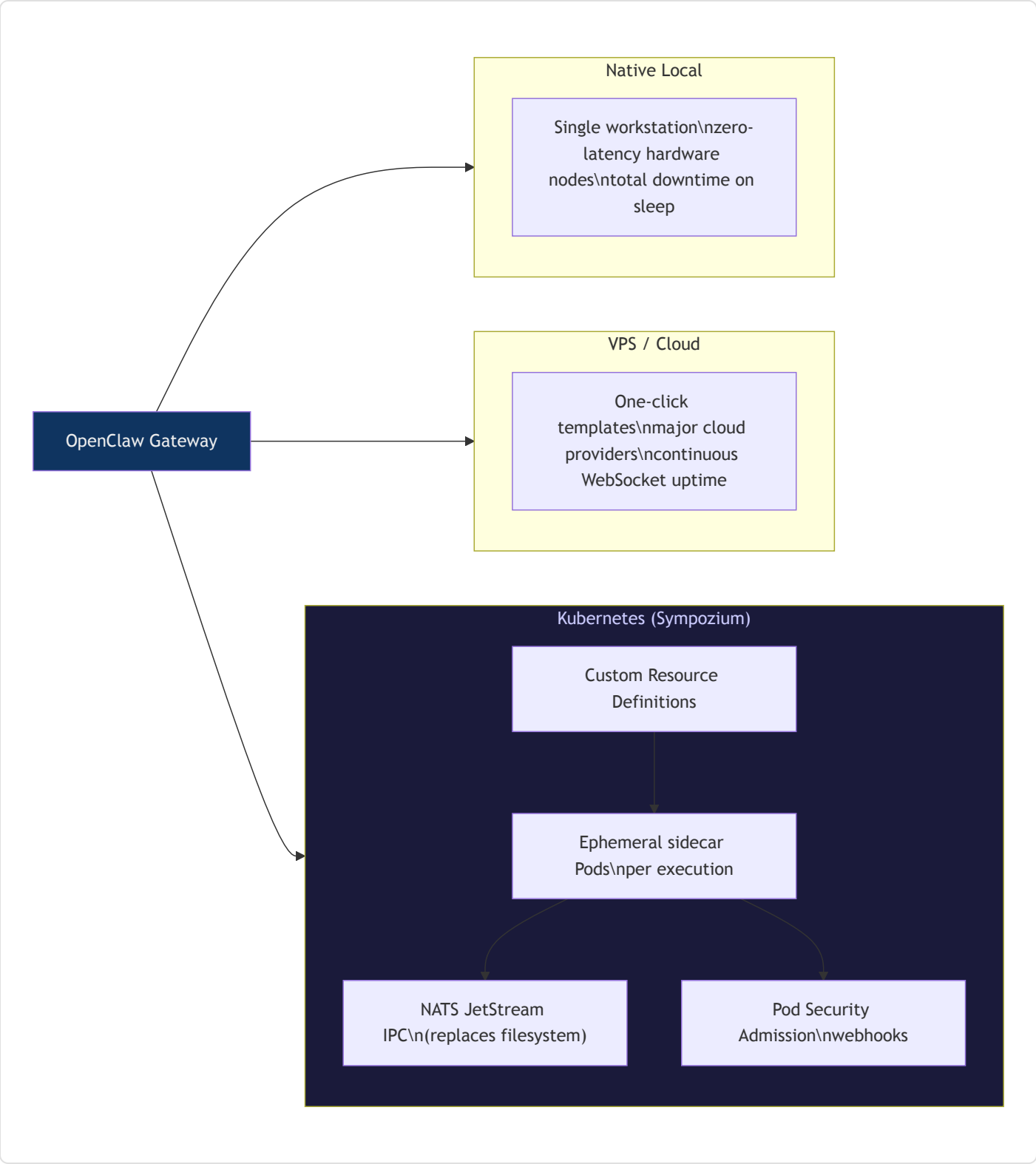

SKILL.md\n(human docs,
injected into context)

7. Deployment Models

NanoClaw



OpenClaw


Native Local
Single workstation\nzero-latency hardware nodes\ntotal downtime on sleep
VPS / Cloud
One-click templates\nmajor cloud providers\ncontinuous WebSocket uptime
OpenClaw Gateway
Kubernetes (Sympozium)
Custom Resource Definitions
Ephemeral sidecar Pods\nper execution
NATS JetStream IPC\n(replaces filesystem)
Pod Security Admission\nwebhooks

8. Channel Support (Current + In-Flight PRs)

Channel	NanoClaw	OpenClaw
Telegram	✅ core	✅ core
Slack	✅ core	✅ core
Discord	✅ skill	✅ extension
WhatsApp	✅ skill	✅ Baileys
Signal	✅ skill (PRs #784, #665)	✅ extension
iMessage	—	✅ BlueBubbles
Feishu / Lark	✅ skills (3 PRs)	✅ heavy (6+ PRs)
Matrix	✅ PR #791	—
Mattermost	✅ PR #546	✅ extension
Google Chat	✅ PR #752	—
QQ / NapCat	✅ PRs #821, #796	—
WeChat Work	—	✅ PR #39511
DingTalk	✅ PR #764	—
Web Chat (browser)	✅ PR #797	✅ core web-ui
IMAP Email	—	✅ PR #39625
CLI	✅ PR #680	✅ core
Hardware nodes (camera/location)	❌	✅ core

---

9. AI Provider Support

Provider	NanoClaw	OpenClaw
Anthropic Claude	✅ primary	✅
Ollama (local)	✅ PR #797	✅
llama.cpp	✅ PR #762	—
Generic LLM / OpenAI-compat	✅ PR #557	✅
Azure OpenAI	—	✅ PR #39540

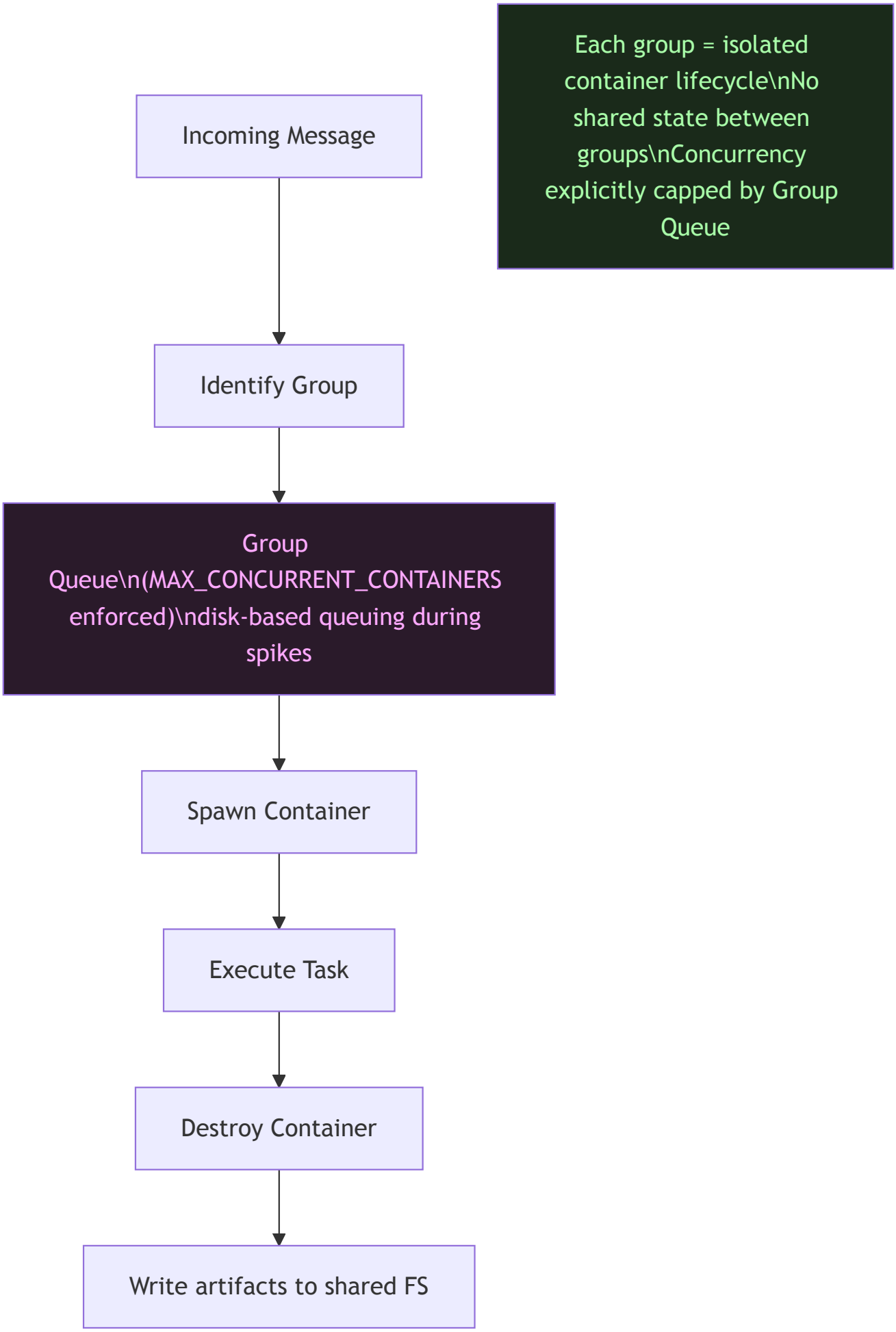
Provider	NanoClaw	OpenClaw
Codex	✅ PR #572	—
Novita AI	—	✅ PR #39675
GitHub Copilot models	—	✅ PR #39613
Moonshot / Kimi	—	✅

---

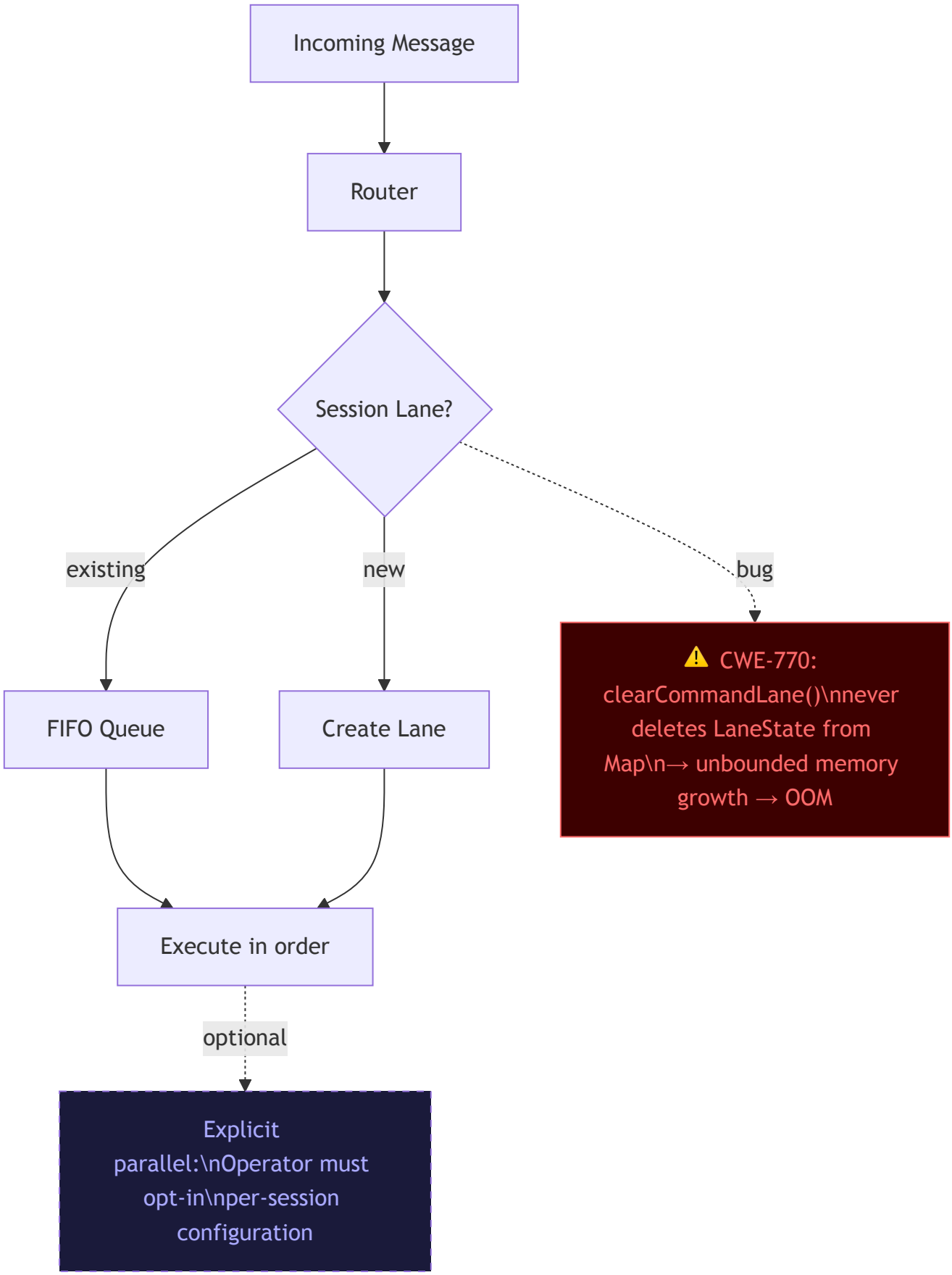
10. Concurrency Model

NanoClaw

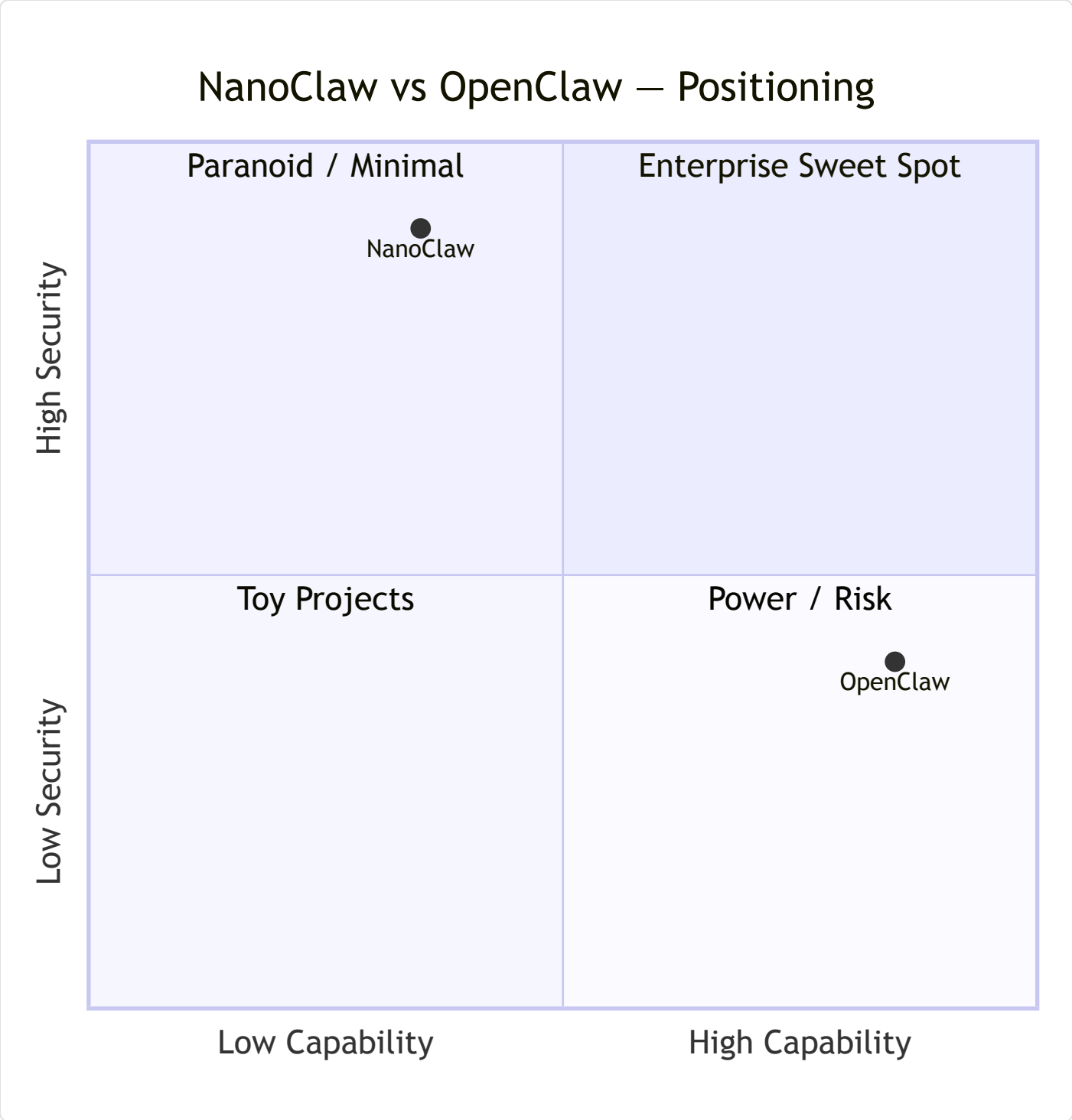
Concurrency is explicitly bounded by the **Group Queue**, which enforces `MAX_CONCURRENT_CONTAINERS` and uses disk-based queuing during load spikes. Each group receives an isolated container lifecycle with no shared state between groups.


Each group = isolated container lifecycle\nNo shared state between groups\nConcurrency explicitly capped by Group Queue
Incoming Message
Identify Group
Group Queue\n(MAX_CONCURRENT_CONTAINERS enforced)\ndisk-based queuing during spikes
Spawn Container
Execute Task
Destroy Container
Write artifacts to shared FS

OpenClaw — Lane Queue (Default Serial, Explicit Parallel)


Incoming Message
Router
Session Lane?
existing
new
bug
FIFO Queue
Create Lane
⚠️ CWE-770: clearCommandLane()\nnever deletes LaneState from Map\n→ unbounded memory growth → OOM
Execute in order
optional
Explicit parallel:\nOperator must opt-in\nper-session configuration

11. What's Actually Different — Summary Matrix



Concern	NanoClaw wins	OpenClaw wins
Security isolation	✅ OS-level, provable	
Attack surface	✅ Minimal, auditable	
Channel breadth		✅ More native integrations
Hardware node support		✅ Camera, location, screen

Concern	NanoClaw wins	OpenClaw wins
Kubernetes / cloud scale		✅ Sympozium
AI provider variety		✅ More providers
Community size		✅ 215k stars
Codebase auditability	✅ 4k lines, LLM-readable	
Skill supply chain safety	✅ Static merge, no runtime exec	
Memory debuggability	✅ Filesystem + SQLite	✅ Markdown edit = instant fix
Real-time streaming	Filesystem-based (lower latency risk)	✅ WebSocket, SSE
Mobile apps	❌	✅ iOS + Android
Canvas / visual workspace	❌	✅ HTML/CSS/JS agent-driven
Multi-tenant safety	❌ (both fail, NanoClaw by design)	❌ (explicit disclaimer)

---

12. Active Development Signals (from open PRs)

NanoClaw (open PR analysis)

- **IPC rewrite** (#816) — moving to JSON-RPC 2.0 over stdio (architectural shift away from filesystem IPC; also changes the threat model: stdio is not cryptographically verified the way the current filesystem JSON path is, so the transition requires careful re-evaluation of the IPC Authorization boundary described in Section 3)
- **Web Chat UI** (#797) — 77k line PR adding browser channel, Cloudflare tunnel, Ollama, Airtable
- **Memory system** (#560/#561) — RAG-based semantic memory being added
- **New channels flood** — QQ, WeChat Work via NapCat, Matrix, Mattermost, Google Chat

OpenClaw (open PR analysis)

- **Feishu mega-PR** (#39496) — 9 streaming bug fixes, 3-layer dedup, calendar CRUD tools
- **IMAP hook** (#39625) — email without Gmail dependency
- **Azure OpenAI + Novita** — provider expansion
- **Wecom channel** (#39511) — WeChat Work already on npm, requesting inclusion
- **Per-agent timezone** (#39610) — cross-cutting config quality-of-life
- **Dual-layer memory** — proposed upgrade from ~65% to higher recall (not yet in PR)

> **Bottom line:** NanoClaw is being pushed toward richer features (web UI, RAG memory, more channels) while trying to maintain its security-first identity; the eight-stage context management investment in OpenClaw signals that long-session reliability is now a first-class concern for that project as well. OpenClaw is maturing

> its core (fixing IPC bugs, better memory, provider breadth). The gap in security model is structural — NanoClaw's container boundary is a design constraint that OpenClaw doesn't share and can't easily retrofit.